Informe de Cierre de Módulo



Oficios Formosa

Plataforma de contratación de trabajadores de oficios

Fecha de emisión	26/06/2026
Elaborado por	Britez Matías — Android Engineer
Versión	v1.1

Informe de Cierre de Módulo

M01 | Infraestructura Técnica | 26/06/2026

1. Alcance entregado

Componente	Descripción
Arquitectura modular	14 módulos Gradle independientes: :app, :core:common, :core:data, :core:designsystem, :core:domain, :core:network, :core:testing, :domain, :feature:auth, :feature:home, :feature:onboarding y :feature:settings. La separación en capas garantiza bajo acoplamiento y construcciones incrementales eficientes.
Inyección de dependencias (Hilt)	Configuración completa con @HiltAndroidApp, @HiltViewModel y módulos Hilt por feature. No se utiliza ningún service locator ni singleton manual; toda dependencia es inyectada y reemplazable en tests.
Retrofit 2 + OkHttp 4	Dos instancias independientes: @ApiRetrofit apunta al backend real (apioficios.pushsoftware.com.ar) y @DemoRetrofit a mocks locales. Timeout global de 30 segundos. SanitizedLoggingInterceptor activo únicamente en modo DEBUG para evitar filtración de tokens en producción.
Room Database	Configuración base de caché local: entidades, DAOs y migraciones definidas. Lista para ser utilizada en los módulos de pedidos, búsqueda y trabajadores.
Navigation Compose	NavHost único centralizado en :app. Funciones de extensión NavGraphBuilder por feature. Constantes *Destination por módulo. Funciones de utilidad: navigateToLoginClearingBackStack(), navigateToInitialDestination() y navigateToOnboardingGate().
AppResult<T>	Interfaz sellada universal para manejo de errores: Success | Error con AppError tipado (Network, Validation, Unauthorized, Forbidden, NotFound, Conflict, Unknown). Todos los repositorios del proyecto retornan este tipo.
AuthInterceptor	Interceptor de OkHttp que inyecta automáticamente el header Authorization: Bearer {token} en todas las solicitudes no marcadas como públicas.
TokenAuthenticator	Implementa OkHttp Authenticator para refrescar el JWT de forma automática. Utiliza Mutex de corrutinas para evitar race conditions cuando múltiples solicitudes paralelas detectan un token vencido simultáneamente. Máximo de 2 reintentos; si falla, dispara cierre de sesión automático.
SessionManager	Singleton que expone authState: Flow<AuthState>. Gestiona restoreFromStorage(), updateTokens() y updateCachedUser(). El token JWT se persiste con EncryptedSharedPreferences (SecurityCrypto 1.1.0-alpha06) para cumplir con las buenas prácticas de seguridad en Android.
PublicEndpoints	Cinco rutas marcadas como públicas que el AuthInterceptor excluye automáticamente del encabezado de autorización: login, registro, recuperación de contraseña, verificación de código y reseteo de contraseña.

2. Decisiones técnicas clave

- Arquitectura modular desde el día 1: la separación entre módulos :core y :feature evita dependencias circulares, acelera las builds incrementales y permite que cada feature sea desarrollada y testeada de forma aislada.
- AppResult<T> como contrato universal: al establecer un tipo de retorno uniforme para todos los repositorios, el manejo de errores es consistente en toda la aplicación y completamente testeable sin dependencias externas.
- Dual Retrofit (@ApiRetrofit / @DemoRetrofit): permite trabajar con mocks locales mientras el backend no está disponible, sin necesidad de modificar el código de ningún feature ni ViewModel.
- TokenAuthenticator con Mutex de corrutinas: garantiza que si dos solicitudes paralelas detectan simultáneamente un token vencido, solo una ejecuta el refresco; las demás esperan y reutilizan el token renovado. Sin este mecanismo, se producirían errores en cadena e invalidaciones de token por carrera de condición.
- EncryptedSharedPreferences para tokens: el JWT no se almacena en SharedPreferences plano. Se cifra en reposo con AES-256-GCM mediante SecurityCrypto, cumpliendo con las directrices de seguridad de Android y los requisitos de la Play Store para datos sensibles.
- SanitizedLoggingInterceptor solo en DEBUG: en builds de release el interceptor de logging está desactivado, lo que impide que tokens o datos de usuarios aparezcan en los registros del sistema.

3. Estado de testing

Componente	Tipo de validación	Notas
AppResult / AppError	Tests unitarios	Cubre casos de éxito, error de red, no autorizado, no encontrado y conflicto.
TokenAuthenticator	Tests unitarios	Refresco exitoso, refresco fallido, máximo de reintentos y Mutex bajo concurrencia.
AuthInterceptor	Tests unitarios	Inyección correcta del token y exclusión de rutas públicas.
SessionManager	Tests unitarios	restoreFromStorage, updateTokens y estados de sesión activa/inactiva.
Navigation (AppRoutes)	Prueba manual	Flujo de navegación entre grafos verificado en dispositivo físico con Android 10, 12 y 14.

4. Dependencias que habilita

- Todos los módulos de feature dependen de este módulo para inyección de dependencias, networking, navegación y manejo de errores. Sin él, ningún otro módulo puede compilar de forma independiente.
- TokenAuthenticator y SessionManager son requisito previo para cualquier pantalla que requiera autenticación, incluyendo los módulos de Home, Onboarding y todas las acciones de usuario.
- AppResult<T> establece el contrato de retorno que deben seguir todos los repositorios presentes y futuros del proyecto, garantizando uniformidad en el tratamiento de errores.

- La configuración de Navigation Compose en este módulo define las convenciones de rutas y extensiones que todos los features deberán seguir al integrar sus grafos de navegación.

Resumen de cierre

Módulo	Días estimados	Días reales	Variación	Estado
M01 — Infraestructura Técnica	4 días	4 días	**Sin variación**	**Completado**